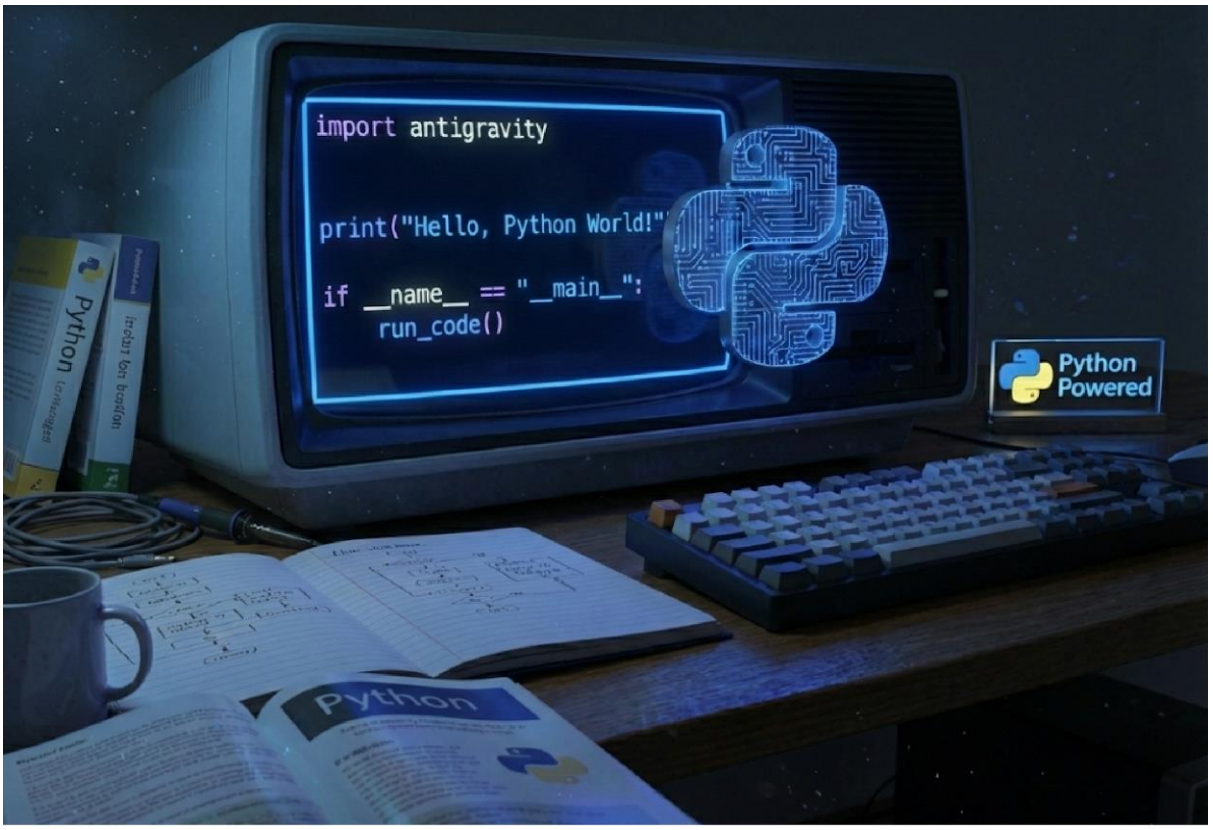
MIT | **Academy of Engineering**

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Subject (Code):
Essentials of Data Science
(2304102L)

EDS CodeTantra Programs

Name	(PRN)	Div- Batch
TANMAY Rahul MUTKULE	(202501090093)	ME2 – M21



```
# Addition

addition_result=matrix_a+matrix_b

print("Addition (A + B):")

print(addition_result)

# Subtraction

substraction_result=matrix_a-matrix_b

print("Subtraction (A - B):")

print(substraction_result)

# Multiplication (element-wise)

elementwise_multiplication_result=matrix_a*matrix_b

print("Element-wise Multiplication (A * B):")

print(elementwise_multiplication_result)

# Matrix multiplication (dot product)

matrix_multiplication_result=np.dot(matrix_a, matrix_b)

print("A dot B:")

print(matrix_multiplication_result)

# Transpose

transpose_a=matrix_a.T

print("Transpose of A:")

print(transpose_a)
```

- **Find the count of students who got more than 90 marks in Subject 2**: Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject**: Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored >=90**: Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order**: Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1**: Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1**: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

# 1. Print all student details

print("All student Details:\n", a )

# 2. print total students

print("Total Students:", a.shape[0])

# 3. Print all student Roll numbers

print("All Student Roll Nos",a[:,0]  )

# 4. Print subject 1 marks

print("Subject 1 Marks", a[:,1])

# 5. print minimum marks of Subject 2
```

.

Practical 04

4.1. Practice Lab Assignments

4.1.1. Pandas – Series Creation and Manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the **`groupby`** and **`mean()`** operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

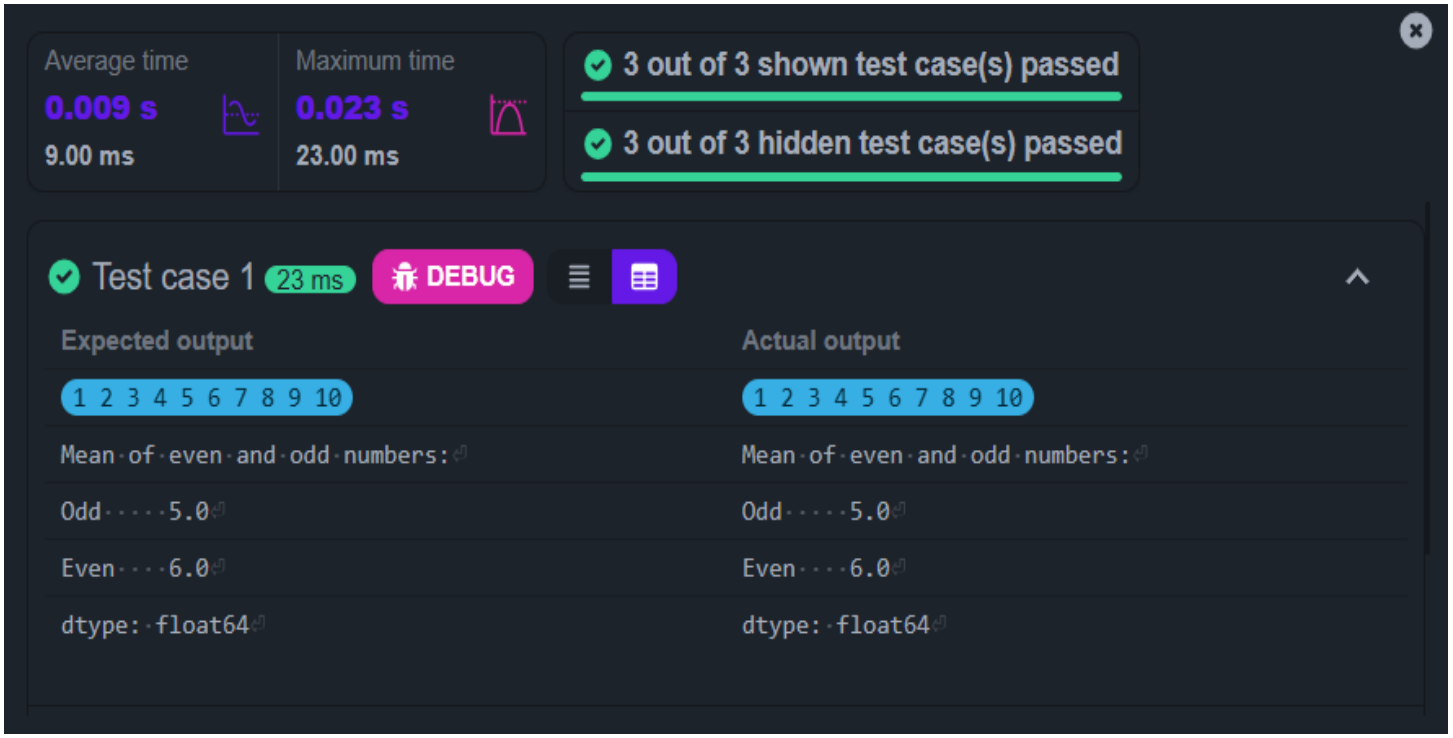
- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

```
import pandas as pd
numbers = list(map(int, input().split()))
series = pd.Series(numbers)
grouped = series.groupby(series%2==0).mean()
grouped.index = ['Even' if is_even else 'Odd' for is_even in
grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```


Average time
0.009 s
9.00 ms
Maximum time
0.023 s
23.00 ms
3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
23 ms
DEBUG
Expected output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64
Actual output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64

4.1.2. Dictionary to DataFrame

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
new_name = input("New name: ")
new_age = int(input("New age: "))
new_row = {'Name': new_name, 'Age': new_age }
df = df.append(new_row,ignore_index = True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
modify_index = int(input("Index of row to modify: "))
new_age = int(input("New age: "))
df.at[modify_index,'Age'] = new_age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
del_index = int(input("Index of row to delete: "))
df = df.drop(del_index).reset_index(drop = True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
genders  = input("Enter genders separated by space: ").split()
df["Gender"] = genders
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name'] = df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df = df.drop(columns = 'Age').reset_index(drop = True)
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

Average time	Maximum time
0.080 s	0.093 s
79.50 ms	93.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 93 ms DEBUG

Expected output	Actual output
Original DataFrame:	Original DataFrame:
Name Age	Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
New name: Susan	New name: Susan
New age: 29	New age: 29
After adding a row:	After adding a row:
Name Age	Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
3 Susan 29	3 Susan 29
Index of row to modify: 0	Index of row to modify: 0
New age: 27	New age: 27
After modifying a row:	After modifying a row:
Name Age	Name Age
0 Alice 27	0 Alice 27
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
3 Susan 29	3 Susan 29
Index of row to delete: 3	Index of row to delete: 3
After deleting a row:	After deleting a row:
Name Age	Name Age
0 Alice 27	0 Alice 27
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
Enter genders separated by space: F M M	Enter genders separated by space: F M M
After adding a new column:	After adding a new column:
Name Age Gender	Name Age Gender

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students(limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold(consider the threshold grade is **'B'**).

Note:
Refer to the displayed test cases for better understanding.

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
"Grade"])

# Display the first five rows
print("First five rows:")
print(data.head())

# Calculate the average age.
average_age = data['Age'].mean()
print(f"Average age: {round(average_age, 2)}")

# Filter students with a grade up to 'B', i.e., 'Grade' >= 'B'
filtered_students = data[data['Grade'] >= 'B']
print("Students with a grade up to B")
print(filtered_students)
```

Average time	Maximum time
0.044 s	0.063 s
44.00 ms	63.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 63 ms DEBUG

Expected output	Actual output
studentdata.txt	studentdata.txt
First five rows:	First five rows:
···Name··Age·Grade	···Name··Age·Grade
0··John···25·····A	0··John···25·····A
1··Alin···22·····B	1··Alin···22·····B
2··Emma···24·····A	2··Emma···24·····A
3··Mike···23·····C	3··Mike···23·····C
4··Sony···21·····B	4··Sony···21·····B
Average age: 23.29	Average age: 23.29
Students with a grade up to B	Students with a grade up to B
···Name··Age·Grade	···Name··Age·Grade
0··John···25·····A	0··John···25·····A
1··Alin···22·····B	1··Alin···22·····B
2··Emma···24·····A	2··Emma···24·····A
4··Sony···21·····B	4··Sony···21·····B
5··Amia···26·····A	5··Amia···26·····A

4.2. Lab Assignments

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

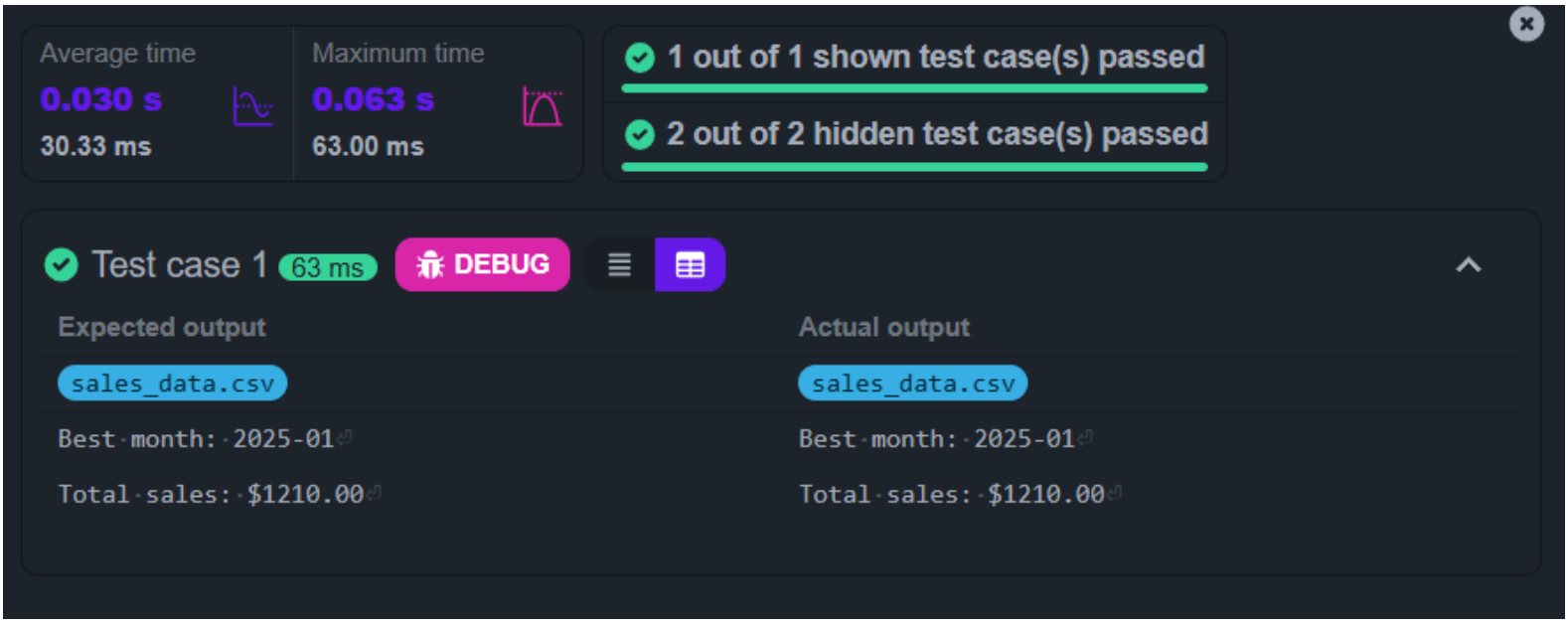
Date, Product, Quantity, Price, City
2025-01-01, Product A, 5, 20, New York
2025-01-01, Product B, 3, 15, Los Angeles
2025-01-02, Product A, 7, 20, New York
2025-01-02, Product C, 4, 30, Chicago
2025-01-03, Product B, 2, 15, Chicago
2025-01-03, Product A, 8, 20, Los Angeles
2025-01-04, Product C, 6, 30, New York
2025-01-04, Product B, 5, 15, Los Angeles
2025-01-05, Product A, 3, 20, Chicago
2025-01-05, Product C, 10, 30, Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Calculate the total sales for each transaction (Quantity * Price)
df['Total Sales'] = df['Quantity'] * df['Price']

# Convert the 'Date' column to datetime objects to extract month
information
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Create a new column for the Month name
df['Month'] = df['Date'].dt.to_period('M')

# Group by Month and sum the Total Sales
monthly_sales = df.groupby('Month')['Total Sales'].sum()

# Find the best month (index with the max sales value) and the sales figure
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

# Display the results
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```



4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

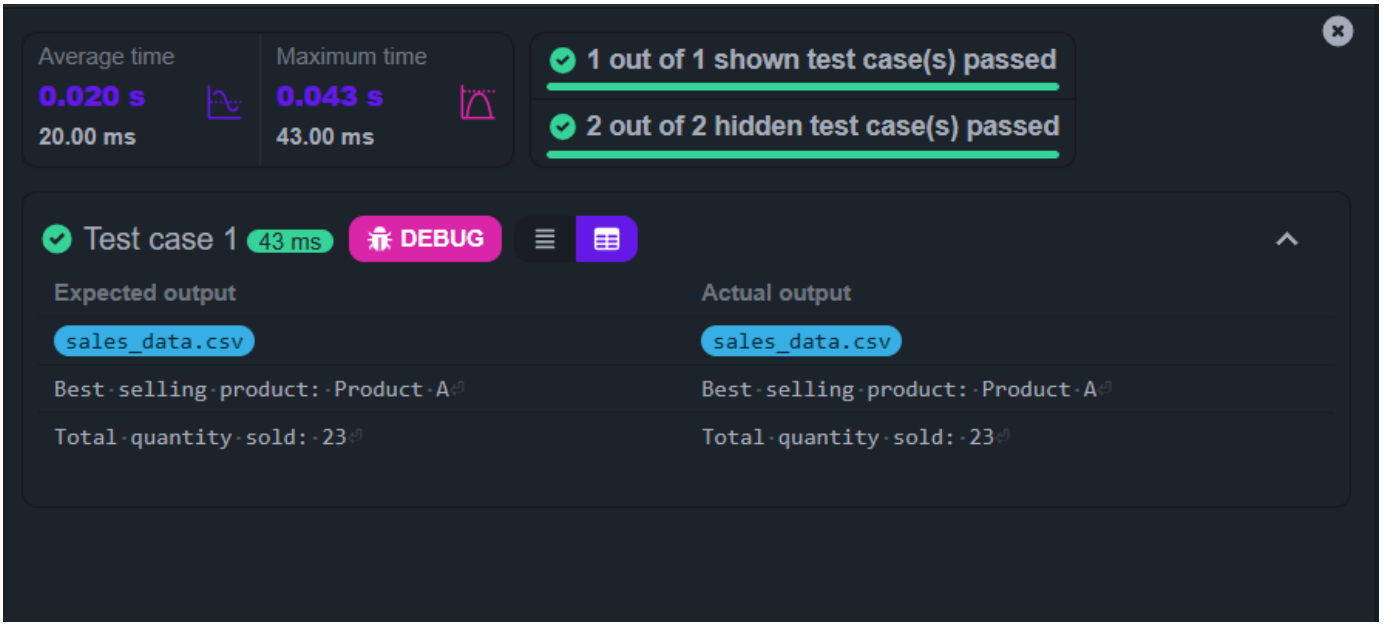
Sample Data:

Date, Product, Quantity, Price, City
2025-01-01, Product A, 5, 20, New York
2025-01-01, Product B, 3, 15, Los Angeles
2025-01-02, Product A, 7, 20, New York
2025-01-02, Product C, 4, 30, Chicago
2025-01-03, Product B, 2, 15, Chicago
2025-01-03, Product A, 8, 20, Los Angeles
2025-01-04, Product C, 6, 30, New York
2025-01-04, Product B, 5, 15, Los Angeles
2025-01-05, Product A, 3, 20, Chicago
2025-01-05, Product C, 10, 30, Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```python
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Group by Product and calculate the total quantity sold for each
product_quantity = df.groupby('Product')['Quantity'].sum()

# Find the most sold product (index with the max total quantity value)
and the total quantity sold of that product
best_product = product_quantity.idxmax()
highest_quantity = product_quantity.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```



4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

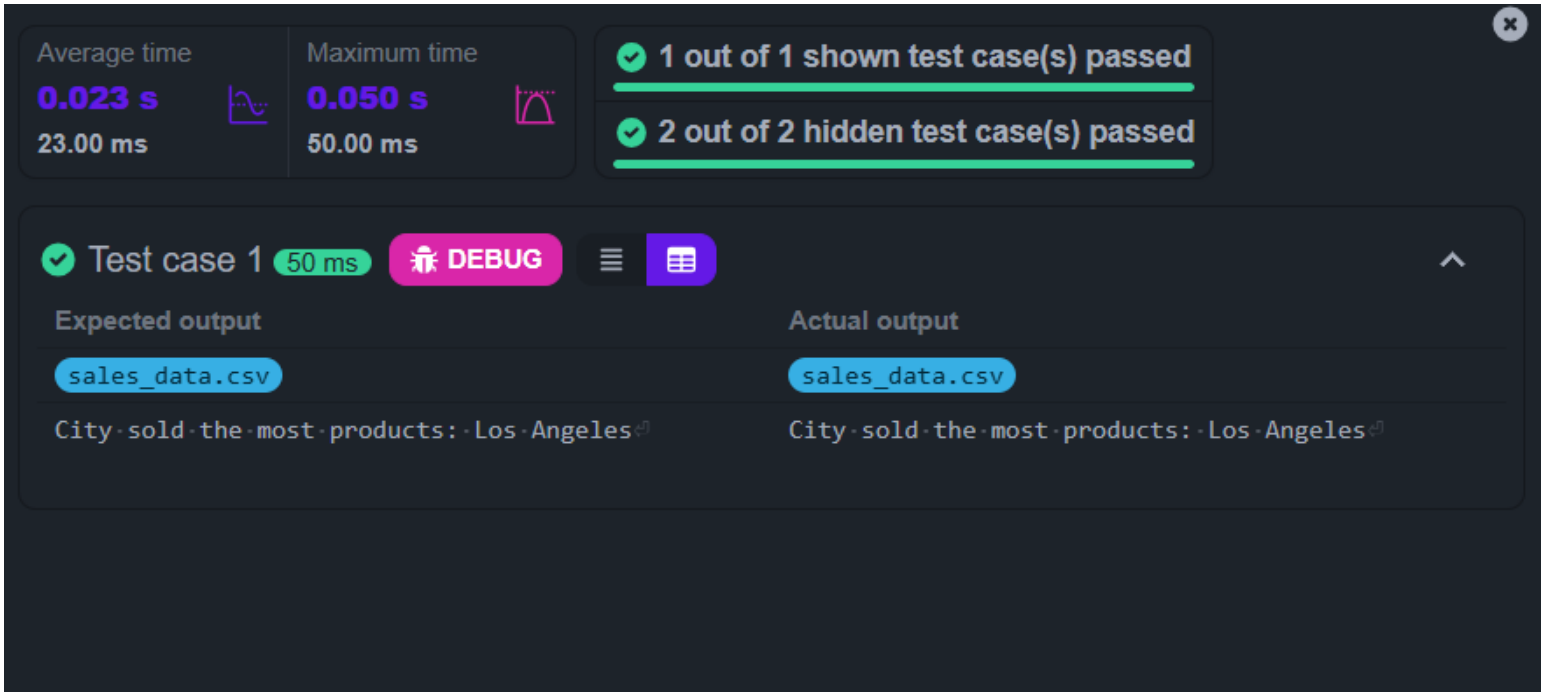
Sample Data:

Date, Product, Quantity, Price, City
2025-01-01, Product A, 5, 20, New York
2025-01-01, Product B, 3, 15, Los Angeles
2025-01-02, Product A, 7, 20, New York
2025-01-02, Product C, 4, 30, Chicago
2025-01-03, Product B, 2, 15, Chicago
2025-01-03, Product A, 8, 20, Los Angeles
2025-01-04, Product C, 6, 30, New York
2025-01-04, Product B, 5, 15, Los Angeles
2025-01-05, Product A, 3, 20, Chicago
2025-01-05, Product C,10,30, Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Group by City and sum the Quantity column
city_sales = df.groupby('City')['Quantity'].sum()

# Find the best city (index with the max total quantity value)
best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```


Average time
0.023 s
23.00 ms
Maximum time
0.050 s
50.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
50 ms
DEBUG
Expected output
sales_data.csv
City sold the most products: Los Angeles
Actual output
sales_data.csv
City sold the most products: Los Angeles

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date, Product, Quantity, Price, City
2025-01-01, Product A, 5, 20, New York
2025-01-01, Product B, 3, 15, Los Angeles
2025-01-02, Product A, 7, 20, New York
2025-01-02, Product C, 4, 30, Chicago
2025-01-03, Product B, 2, 15, Chicago
2025-01-03, Product A,8, 20, Los Angeles
2025-01-04, Product C,6, 30, New York
2025-01-04, Product B, 5, 15, Los Angeles
2025-01-05, Product A, 3, 20, Chicago
2025-01-05, Product C, 10, 30, Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B**: Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C**: Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C**: Appears in transactions on 2025-01-04.

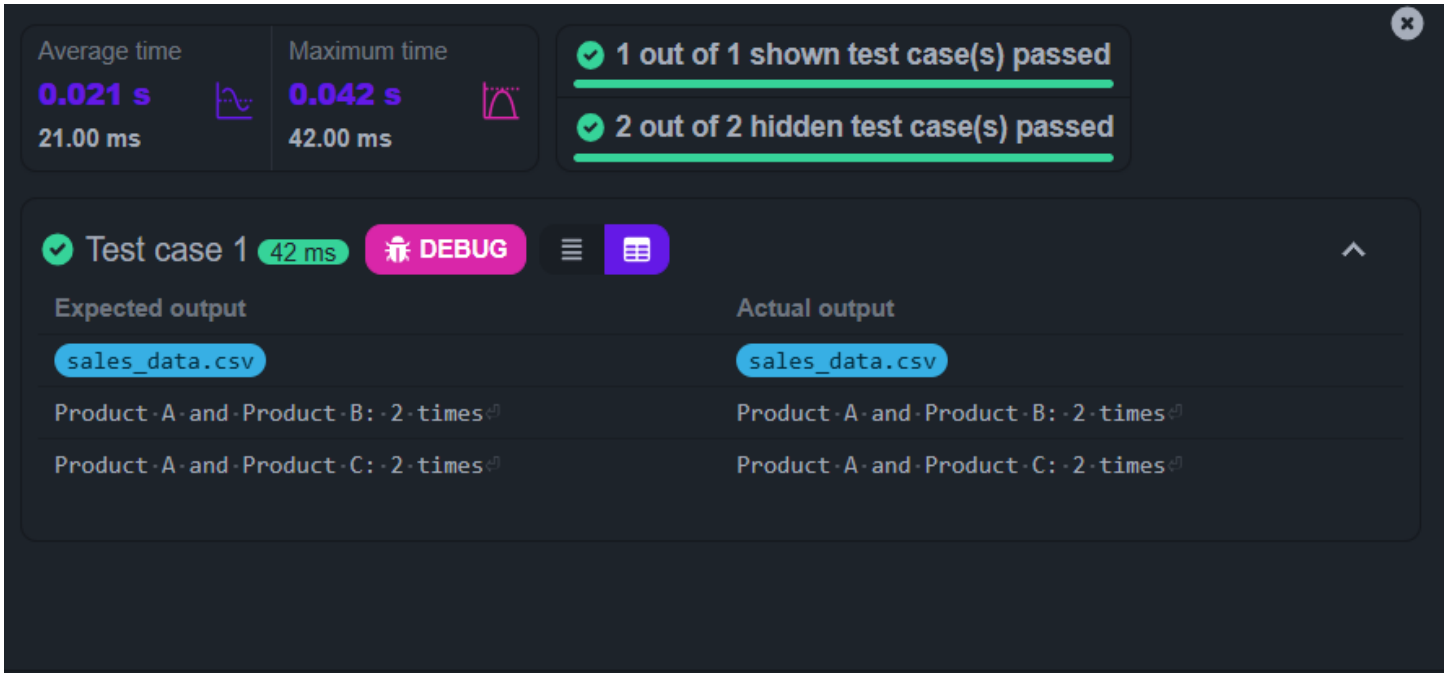
Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()
# Read data from the specified CSV file
df = pd.read_csv(file_name)


# write the code
daily_transactions=df.groupby('Date')['Product'].apply(list)
pair_counts=Counter()
for products in daily_transactions:
    products=sorted(products)
    pairs=list(combinations(products, 2))
    pair_counts.update(pairs)

# Output the most frequent product pairs
if pair_counts:
    max_count=max(pair_counts.values())
    for pair, count in pair_counts.items():
        if count==max_count:
            print(f"{pair[0]} and {pair[1]}: {count} times")
else:
    print("No product pairs found.")
```



4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SipSp 'Parch' column

The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

- PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
- 1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, , S
- 2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
- 3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2. 3101282, 7.925, , S
- 4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, C123, S
- 5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, , S
- 6, 0, 3, "Moran, Mr. James", male, , 0, 0, 330877, 8.4583, , Q
- 7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17463, 51.8625, E46, S
- 8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 3, 1, 349909, 21.075, , S
- 9, 1, 3, "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 2, 347742, 11.1333, , S
- 10, 1, 2, "Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 237736, 30.0708, , C

Note: Refer to the visible test case for better reference.

```python
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())
# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
data['Age'].fillna(data['Age'].median(), inplace = True)

# 8. Fill missing values in the 'Embarked' column with the mode
data['Embarked'].fillna(data['Embarked'].mode()[0],inplace=True)

# 9. Drop the 'Cabin' column due to many missing values
data.drop(columns='Cabin',inplace = True)

# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
data["FamilySize"] = data['SibSp']+data['Parch']
```

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Sample Data:

- PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
- 1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, , S
- 2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
- 3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2. 3101282, 7.925, , S
- 4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, C123, S
- 5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, , S
- 6, 0, 3, "Moran, Mr. James", male, , 0, 0, 330877, 8.4583, , Q
- 7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17463, 51.8625, E46, S
- 8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 3, 1, 349909, 21.075, , S
- 9, 1, 3, "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 2, 347742, 11.1333, , S
- 10, 1, 2, "Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 237736, 30.0708, , C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# Create a new column 'IsAlone' which is 1 if the passenger is alone
(i.e., FamilySize = 0), otherwise 0
data['IsAlone'] = np.where(data['FamilySize'] >= 0, 1, 0)
# data['IsAlone'] = (data['FamilySize'] >= 0).astype(int) # Alternate
way

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'])

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print( mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print( median_fare)

# 6. Get the number of passengers by class
print( data['Pclass'].value_counts())

# 7. Get the number of passengers by gender
print( data['Sex'].value_counts())

# 8. Get the number of passengers by survival status
print( data['Survived'].value_counts())

# 9. Calculate the overall survival rate
survival_rate = data['Survived'].mean()
print(format(survival_rate))

# 10. Calculate the survival rate by gender
survival_by_gender = data.groupby('Sex')['Survived'].mean()
```

Average time	Maximum time
0.084 s	0.084 s
84.00 ms	84.00 ms

1 out of 1 shown test case(s) passed

Test case 1 84 ms DEBUG

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3····491	3····491
1····216	1····216
2····184	2····184
Name:·Pclass,·dtype:·int64	Name:·Pclass,·dtype:·int64
0····577	0····577
1····314	1····314
Name:·Sex,·dtype:·int64	Name:·Sex,·dtype:·int64
0····549	0····549
1····342	1····342
Name:·Survived,·dtype:·int64	Name:·Survived,·dtype:·int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0····0.188908	0····0.188908
1····0.742038	1····0.742038
Name:·Survived,·dtype:·float64	Name:·Survived,·dtype:·float64

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Sample Data:

- PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
- 1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, , S
- 2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
- 3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2. 3101282, 7.925, , S
- 4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, C123, S
- 5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, , S
- 6, 0, 3, "Moran, Mr. James", male, , 0, 0, 330877, 8.4583, , Q
- 7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17463, 51.8625, E46, S
- 8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 3, 1, 349909, 21.075, , S
- 9, 1, 3, "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 2, 347742, 11.1333, , S
- 10, 1, 2, "Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 237736, 30.0708, , C

Note: Refer to the visible test case for better reference.

```python
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarkation location (Embarked_S)
print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by passenger class
print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by passenger class

print(data.groupby('Pclass')['Age'].mean())

# 7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())

# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())

# 9. Get the number of survivors by class
# Filter for Survived=1, then count occurrences of Pclass
print(data[data['Survived'] >= 1]['Pclass'].value_counts())

# 10. Get the number of non-survivors by class
# Filter for Survived=0, then count occurrences of Pclass
print(data[data['Survived'] >= 0]['Pclass'].value_counts())
```

Average time
0.099 s
99.00 ms

Maximum time
0.099 s
99.00 ms

1 out of 1 shown test case(s) passed

Test case 1 99 ms DEBUG

Expected output	Actual output
Pclass	Pclass
1····0.629630	1····0.629630
2····0.472826	2····0.472826
3····0.242363	3····0.242363
Name:·Survived,·dtype:·float64	Name:·Survived,·dtype:·float64
Embarked_S	Embarked_S
0····0.506073	0····0.506073
1····0.336957	1····0.336957
Name:·Survived,·dtype:·float64	Name:·Survived,·dtype:·float64
FamilySize	FamilySize
0·····0.303538	0·····0.303538
1·····0.552795	1·····0.552795
2·····0.578431	2·····0.578431
3·····0.724138	3·····0.724138
4·····0.200000	4·····0.200000
5·····0.136364	5·····0.136364
6·····0.333333	6·····0.333333
7·····0.000000	7·····0.000000
10····0.000000	10····0.000000

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Sample Data:

- PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
- 1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, , S
- 2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
- 3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2. 3101282, 7.925, , S
- 4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, C123, S
- 5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, , S
- 6, 0, 3, "Moran, Mr. James", male, , 0, 0, 330877, 8.4583, , Q
- 7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17463, 51.8625, E46, S
- 8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 3, 1, 349909, 21.075, , S
- 9, 1, 3, "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 2, 347742, 11.1333, , S
- 10, 1, 2, "Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 237736, 30.0708, , C

Note: Refer to the visible test case for better reference.

```python
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# 1. Get the number of survivors by gender
print(data[data['Survived']>=1]['Sex'].value_counts())

# 2. Get the number of non-survivors by gender
print(data[data['Survived']>=0]['Sex'].value_counts())

# 3. Get the number of survivors by embarked location
print(data[data['Survived']>=1]['Embarked_S'].value_counts())

# 4. Get the number of non-survivors by embarked location
print(data[data['Survived']>=0]['Embarked_S'].value_counts())

# 5. Calculate the percentage of children (Age < 18) who survived
print(data[data['Age']<18]['Survived'].mean())

# 6. Calculate the percentage of adults (Age >= 18) who survived
print(data[data['Age']>=18]['Survived'].mean())

# 7. Get the median age of survivors
print(data[data['Survived']>=1]['Age'].median())

# 8. Get the median age of non-survivors
print(data[data['Survived']>=0]['Age'].median())

# 9. Get the median fare of survivors
print(data[data['Survived']>=1]['Fare'].median())

# 10. Get the median fare of non-survivors
print(data[data['Survived']>=0]['Fare'].median())
```

Average time
0.108 s
108.00 ms

Maximum time
0.108 s
108.00 ms

1 out of 1 shown test case(s) passed

Test case 1 108 ms DEBUG

Expected output	Actual output
female····233	female····233
male·····109	male·····109
Name:·Sex,·dtype:·int64	Name:·Sex,·dtype:·int64
male·····468	male·····468
female····81	female····81
Name:·Sex,·dtype:·int64	Name:·Sex,·dtype:·int64
1····217	1····217
0····125	0····125
Name:·Embarked_S,·dtype:·int64	Name:·Embarked_S,·dtype:·int64
1····427	1····427
0····122	0····122
Name:·Embarked_S,·dtype:·int64	Name:·Embarked_S,·dtype:·int64
0.5398230088495575	0.5398230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0
26.0	26.0
10.5	10.5